

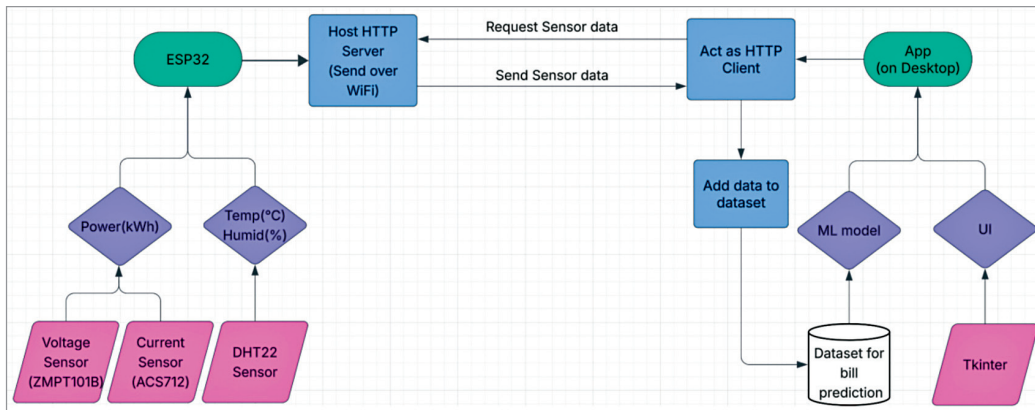


Fig. 2: System flowchart

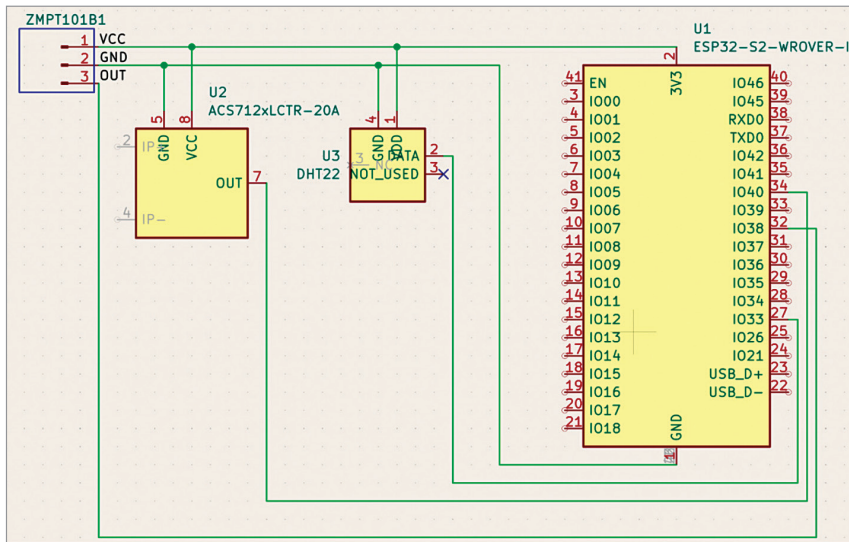


Fig. 3: Circuit diagram

```

void loop() {
  server.handleClient();

  // --- Current & Voltage ---
  Voltage = getVPP();
  VRMS = (Voltage / 2.0) * 0.707;
  AmpsRMS = (VRMS * 1000) / mVperAmp;
  Watt = (AmpsRMS * 240 / 1.3);
  kWh = (Watt / 1000.0) * (10.0 / 3600.0); // 10ms sample window
  energy += kWh;

  // --- Temperature & Humidity ---
  if (sampleCount < MAX_SAMPLES) {
    float t = dht.readTemperature();
    float h = dht.readHumidity();
    if (!isnan(t) && !isnan(h)) {
      tempSamples[sampleCount] = t;
      humiditySamples[sampleCount] = h;
      sampleCount++;
    }
  }

  struct tm timeinfo;
  if (getLocalTime(&timeinfo)) {
    int currentHour = timeinfo.tm_hour;
    int currentMinute = timeinfo.tm_min;
    int currentSecond = timeinfo.tm_sec;
    int currentDay = timeinfo.tm_mday;

    if (currentHour == 23 && currentMinute == 59 && currentSecond == 59 && currentDay != lastLoggedDay) {
      char dateStr[20];
      strftime(dateStr, sizeof(dateStr), "%d/%m/%Y", &timeinfo);
    }
  }
}

```

Fig. 4: Snippet of ESP32 code

application by hosting a web server on the ESP32, where the app functions as a client. All programming code is available via the EFY QR code provided at the end of the write-up.

The ESP32 continuously collects real-time power, temperature, and humidity data and calculates cumulative

energy usage (kWh). At 11:59:59 PM each day, it computes the median temperature and humidity, logs the total daily energy consumption, and stores this information in its onboard memory. When the desktop app is opened, pressing the 'Sync ESP32 Logs' button retrieves all stored logs and keeps the dataset up to date. Fig. 4 shows the snippet of ESP32 code.

This process ensures accurate bill prediction and complete historical energy tracking. The app also displays live statistics such as current power usage, temperature, and humidity, making it a comprehensive Wi-Fi-enabled energy monitoring solution.

Once the ESP32 connects to Wi-Fi, the serial monitor displays messages similar to Fig. 5, confirming the connection and the start of the HTTP server. The desktop user interface (UI) is created using Tkinter. Fig. 6 shows a sample of the UI, while Fig. 7 presents a snippet of Tkinter code.

The UI acts as a client application that connects to the ESP32 over Wi-Fi to fetch real-time data such as current power usage, room temperature, and humidity, presenting it in a clear and readable format. All

```

.....
WiFi connected:
IP ADDRESS
HTTP server started

```

Fig. 5: Connection confirmation